IR-Based Driver Drowsiness Detection & Alarm System

Expanded Arduino code, detailed explanation, algorithm, testing guide and viva questions

Hardware: IR sensor D2 · Buzzer D8 · Red LED D9 · Yellow LED D10 · Green LED D11

Timing: Warning at 1 second · Drowsiness alert at 4 seconds · Danger alert at 8 seconds

Updated Arduino code

Copy the complete sketch into Arduino IDE. It replaces the earlier sketch; do not add a second setup() or loop(). The code assumes LOW means eye closed.

```
// IR-Based Driver Drowsiness Detection and Alarm System
// Sensor assumption: LOW = eye closed; HIGH = eye open.
// Hardware: IR sensor D2, buzzer D8, red LED D9,
//           yellow LED D10, green LED D11.
// Prototype only: not a certified vehicle safety system.

const int IR_SENSOR = 2;
const int BUZZER = 8;
const int RED_LED = 9;
const int YELLOW_LED = 10;
const int GREEN_LED = 11;

const unsigned long WARNING_TIME = 1000; // 1 second
const unsigned long DROWSY_TIME  = 4000; // 4 seconds
const unsigned long DANGER_TIME  = 8000; // 8 seconds

unsigned long eyeClosedStart = 0;
unsigned long lastStatusTime = 0;

bool eyeClosed = false;
int alertLevel = 0; // 0 normal, 1 warning, 2 drowsy, 3 danger

void setLEDs(bool green, bool yellow, bool red) {
  digitalWrite(GREEN_LED, green ? HIGH : LOW);
  digitalWrite(YELLOW_LED, yellow ? HIGH : LOW);
  digitalWrite(RED_LED, red ? HIGH : LOW);
}

void stopAlarm() {
  noTone(BUZZER);
}

void setup() {
  pinMode(IR_SENSOR, INPUT);
  pinMode(BUZZER, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);

  setLEDs(false, false, false);
  stopAlarm();

  Serial.begin(9600);
  Serial.println("IR Driver Drowsiness Detection System");
  Serial.println("Startup test...");

  // Check that LEDs and buzzer respond.
  setLEDs(true, false, false);
  delay(250);
  setLEDs(false, true, false);
  delay(250);
  setLEDs(false, false, true);
  tone(BUZZER, 1500, 250);
  delay(300);

  stopAlarm();
  setLEDs(false, false, false);
  Serial.println("System ready.");
}

void loop() {
  int sensorValue = digitalRead(IR_SENSOR);
  unsigned long now = millis();

  if (sensorValue == LOW) { // Eye closed for this sensor setup
    if (!eyeClosed) {
      eyeClosed = true;
      eyeClosedStart = now;
      alertLevel = 0;
      Serial.println("Eye closure detected.");
```

```
    }

    unsigned long closedDuration = now - eyeClosedStart;

    if (closedDuration >= DANGER_TIME) {
      if (alertLevel != 3) {
        alertLevel = 3;
        Serial.println("DANGER: prolonged eye closure!");
      }
      setLEDs(false, false, true);
      tone(BUZZER, 2500);
    }
    else if (closedDuration >= DROWSY_TIME) {
      if (alertLevel != 2) {
        alertLevel = 2;
        Serial.println("ALERT: drowsiness detected!");
      }
      setLEDs(false, false, true);
      tone(BUZZER, 1800);
    }
    else if (closedDuration >= WARNING_TIME) {
      if (alertLevel != 1) {
        alertLevel = 1;
        Serial.println("WARNING: possible fatigue.");
      }
      setLEDs(false, true, false);
      tone(BUZZER, 1000, 150);
    }
    else {
      setLEDs(true, false, false);
      stopAlarm();
    }
  }
  else { // Eye open
    if (eyeClosed) {
      Serial.println("Eye reopened. Timer reset.");
    }

    eyeClosed = false;
    eyeClosedStart = 0;
    alertLevel = 0;
    stopAlarm();
    setLEDs(true, false, false);
  }

  if (now - lastStatusTime >= 5000) {
    lastStatusTime = now;
    Serial.print("Status: ");
    Serial.println(eyeClosed ? "EYE CLOSED" : "EYE OPEN");
  }

  delay(20);
}
```

Project explanation and documentation

1. Introduction

The IR-Based Driver Drowsiness Detection and Alarm System is an Arduino-based prototype that demonstrates how a sensor signal can be used to monitor a possible eye-closure event and activate an alert.

The main idea is to measure how long the sensor continuously reports a closed-eye state. A brief event is treated differently from a longer event. LEDs show the current level, and a buzzer provides an audible warning.

2. Problem statement

Driver fatigue can reduce attention and reaction time. If a driver becomes drowsy, a warning may help draw attention to the situation. The purpose of this prototype is to demonstrate a simple low-cost warning sequence using an IR sensor, Arduino, LEDs and a buzzer.

This prototype does not prove that a driver is sleepy: it responds to the sensor's electrical output. A real system must accurately detect the driver's eyes and be tested under different lighting, distance and movement conditions.

3. Objectives

Detect a sensor state that is configured to represent eye closure.

Measure continuous closure duration using the Arduino's millis() timer.

Give a warning at 1 second, a drowsiness alert at 4 seconds, and a danger alert at 8 seconds.

Use LEDs and buzzer tones to communicate alert levels.

Reset the alarm and timer when the sensor returns to the eye-open state.

Display useful messages in the Serial Monitor for demonstration and debugging.

4. Components and their functions

Arduino board: reads the sensor input, measures time, and controls outputs according to the program.

IR sensor module: emits or receives infrared light and produces a digital output based on its design and adjustment. The exact sensor must be checked to confirm which output means open or closed.

Buzzer: creates sound at different frequencies for warning and alert stages. The sketch uses tone() and noTone().

Green LED: indicates the normal state in this sketch.

Yellow LED: indicates the first warning stage.

Red LED: indicates a longer closure and higher alert stages.

Resistors and wiring: use suitable current-limiting resistors for LEDs, common ground, and a stable supply suitable for the board and modules.

5. Pin connections

IR sensor digital output → Arduino D2; buzzer signal → D8; red LED → D9; yellow LED → D10; green LED → D11. Connect sensor power and ground according to its datasheet, and connect all circuit grounds together.

Do not connect a vehicle's 12 V supply directly to an Arduino input or 5 V pin. For a vehicle-powered prototype, use a suitable regulated power converter and proper protection.

6. Working principle

First, the Arduino configures the sensor pin as input and the buzzer and LEDs as outputs. A short startup test lights each LED and briefly sounds the buzzer.

During normal operation, the Arduino repeatedly reads the sensor. When the input first becomes LOW, the program records the start time. It then calculates the elapsed closure time on every loop.

If the closure reaches 1 second, the yellow LED and a short beep provide a warning. At 4 seconds, the red LED and a continuous tone indicate drowsiness. At 8 seconds, the red LED remains on and the buzzer uses a higher frequency for the danger stage.

When the input returns HIGH, the code assumes the eye has reopened. It stops the sound, turns on the green LED, clears the timer state, and waits for the next event.

7. Algorithm

Start the system and configure input/output pins.

Run a brief LED and buzzer test.

Read the digital output of the IR sensor.

If the output first indicates closure, save the current millis() value as the start time.

Calculate closure duration = current millis() – start time.

Choose the alert stage using the 1-second, 4-second and 8-second thresholds.

If the eye-open state returns, stop the buzzer and reset the timer and LEDs.

Repeat continuously while the board has power.

8. Explanation of important code

const int: stores fixed pin numbers, making the wiring easy to change in one place.

const unsigned long: stores timing thresholds in milliseconds. 1000 ms = 1 second, 4000 ms = 4 seconds, and 8000 ms = 8 seconds.

pinMode(): configures each pin as INPUT or OUTPUT.

digitalRead(): reads the sensor's HIGH or LOW logic level.

millis(): returns the number of milliseconds since the Arduino program started. Subtracting the start value measures elapsed time without waiting several seconds in a blocking delay.

eyeClosed: remembers whether the current closure event has already started, so the start time is captured only once per event.

alertLevel: remembers which stage has already been announced. This helps prevent the same Serial Monitor message from being printed on every loop iteration.

setLEDs(): is a helper function that sets the three LEDs in one call.

tone() / noTone(): start and stop the buzzer signal. In this sketch, a tone without a duration continues until stopped or changed.

Serial.begin(9600): opens serial communication. Choose 9600 baud in the Serial Monitor to read the messages.

delay(20): adds a short 20 ms pause between sensor reads. It is not the eye-closure timer; millis() is used for that.

9. Timing table

0 to less than 1 second: green LED; buzzer off.

1 to less than 4 seconds: yellow LED; short warning beep is requested during the loop.

4 to less than 8 seconds: red LED; continuous 1800 Hz tone.

8 seconds or more: red LED; continuous 2500 Hz tone.

Sensor returns to open state: green LED; buzzer off; timer reset.

10. How to test the project

1. Test on a desk, not while driving. Upload the sketch and open Serial Monitor at 9600 baud.

2. Confirm the startup test lights the LEDs and briefly sounds the buzzer.

3. Watch the sensor output while presenting and removing an object or using the intended sensor target. Verify that LOW truly corresponds to the state you want to call eye closed.

4. Keep the sensor in the closure state for less than 1 second; the program should not enter the warning stage.

5. Keep the closure state for more than 1 second, 4 seconds and 8 seconds to check the staged outputs.

6. Return the sensor to the open state. The buzzer should stop, the green LED should turn on, and a reset message should appear.

7. If behavior is reversed, verify the module's logic and adjust the condition. If readings flicker, improve sensor placement and consider appropriate filtering only after confirming the hardware.

11. Advantages

Low-cost demonstration using common electronics.

Simple logic that is easy to explain and modify.

Different LED and sound levels make the alert state visible and audible.

Timer resets when the sensor returns to the normal state.

Serial messages help with testing and troubleshooting.

12. Limitations and future improvements

A basic reflective or obstacle IR sensor does not reliably recognize eyelids. It can react to distance, reflections, movement and ambient conditions, so this circuit alone cannot accurately determine driver drowsiness.

Future development could use a suitable near-infrared camera and validated eye-state detection software, then connect the result to the Arduino or a more capable processor. Additional work could include sensor-failure detection, calibration, filtered readings, event logging and controlled testing.

Do not use this prototype as a real road-safety device. A buzzer cannot guarantee that a driver wakes up, and the project is not a substitute for rest or safe driving practices.

13. Conclusion

This project demonstrates a basic sensor-timing-alert workflow. The Arduino reads a digital sensor, measures the length of a continuous event, and uses LEDs and a buzzer to communicate increasing alert levels. It is suitable for learning and a classroom prototype, but reliable real-world drowsiness detection requires more capable sensing and validation.

14. Viva questions and short answers

Q1. What is the purpose of the project?
To demonstrate an alert when a sensor input indicates a prolonged eye-closure event.

Q2. Why is Arduino used?
It reads the sensor, measures time, and controls the LEDs and buzzer.

Q3. What does millis() do?
It returns milliseconds since the program started and helps calculate elapsed time.

Q4. Why are three LEDs used?
They show normal, warning and high-alert states clearly.

Q5. What happens when the eye opens?
The alarm stops, the green LED turns on, and the timer state resets.

Q6. What does tone() do?
It generates a tone on the buzzer output; noTone() stops it.

Q7. Is the system ready for use in a real vehicle?
No. It is a prototype and the sensor cannot reliably identify eyelid closure in all conditions.